PAYROLL MANAGEMENT SYSTEM

Class XII Computer Science Project
Python + MySQL Connectivity
Student: Junaid Karimi
DR. SONEY LAL PATEL PUBLIC SCHOOL, KANPUR
Academic Session: 2026-2027

```
# ============================================================
# PAYROLL MANAGEMENT SYSTEM
# Class XII Computer Science Project
# Python + MySQL Connectivity
# Student: Junaid Karimi
# School: DR. SONEY LAL PATEL PUBLIC SCHOOL, KANPUR
# Academic Session: 2026-2027
# ============================================================

import mysql.connector
from mysql.connector import Error
from datetime import datetime

# ------------------- DATABASE CONNECTION -------------------
def get_connection():
    try:
        con = mysql.connector.connect(
            host="localhost",
            user="root",
            password="",          # <-- ████ ████ MySQL password ████ (████ ████ ████ ██)
            database="payroll_db"
        )
        return con
    except Error as e:
        print("Database connection error:", e)
        return None

# ------------------- CREATE DATABASE & TABLE -------------------
def setup_database():
    try:
        # ████ ████ database ██ connect ███
        con = mysql.connector.connect(
            host="localhost",
            user="root",
            password=""          # <-- password ████ ██ ████
        )
        cur = con.cursor()
        cur.execute("CREATE DATABASE IF NOT EXISTS payroll_db")
        cur.execute("USE payroll_db")

        cur.execute("""
            CREATE TABLE IF NOT EXISTS employee (
                emp_id INT PRIMARY KEY,
                name VARCHAR(50) NOT NULL,
                designation VARCHAR(30),
                department VARCHAR(30),
                phone VARCHAR(15),
                joining_date DATE,
                basic_salary DECIMAL(10,2),
                da DECIMAL(10,2),
                hra DECIMAL(10,2),
                pf DECIMAL(10,2),
                tax DECIMAL(10,2),
                gross DECIMAL(10,2),
                net DECIMAL(10,2)
            )
        """)
        con.commit()
        print("Database and table ready.")
        cur.close()
        con.close()
    except Error as e:
        print("Setup error:", e)

# ------------------- SALARY CALCULATION -------------------
def calculate_salary(basic, designation):
    des = designation.lower()

    if "manager" in des or "admin" in des:
        da_per, hra_per, pf_per, tax_per = 0.40, 0.25, 0.12, 0.10
    elif any(x in des for x in ["officer", "accountant", "engineer"]):
        da_per, hra_per, pf_per, tax_per = 0.30, 0.20, 0.12, 0.08
    else:
        da_per, hra_per, pf_per, tax_per = 0.20, 0.15, 0.12, 0.05

    da = round(basic * da_per, 2)
    hra = round(basic * hra_per, 2)
    pf = round(basic * pf_per, 2)
    tax = round(basic * tax_per, 2)
    gross = round(basic + da + hra, 2)
    net = round(gross - pf - tax, 2)
    return da, hra, pf, tax, gross, net
```

```
# ------------------- 1. ADD EMPLOYEE -------------------
def add_employee():
    con = get_connection()
    if not con:
        return
    cur = con.cursor()
    try:
        emp_id = int(input("Enter Employee ID: "))
        name = input("Enter Name: ")
        designation = input("Enter Designation: ")
        department = input("Enter Department: ")
        phone = input("Enter Phone: ")
        joining_date = input("Enter Joining Date (YYYY-MM-DD): ")
        basic = float(input("Enter Basic Salary: "))

        da, hra, pf, tax, gross, net = calculate_salary(basic, designation)

        query = """
            INSERT INTO employee
            (emp_id, name, designation, department, phone, joining_date,
             basic_salary, da, hra, pf, tax, gross, net)
            VALUES (%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)
        """
        values = (emp_id, name, designation, department, phone, joining_date,
                  basic, da, hra, pf, tax, gross, net)

        cur.execute(query, values)
        con.commit()
        print("\nEmployee added successfully!")
        print(f"Gross: {gross}   Net Salary: {net}")
    except Error as e:
        print("Error while adding:", e)
    finally:
        cur.close()
        con.close()

# ------------------- 2. VIEW ALL EMPLOYEES -------------------
def view_all():
    con = get_connection()
    if not con:
        return
    cur = con.cursor()
    try:
        cur.execute("SELECT emp_id, name, designation, department, basic_salary, net FROM employee ORDER BY emp_id ASC")
        rows = cur.fetchall()
        if not rows:
            print("No records found.")
            return
        print("\n{:<8} {:<20} {:<18} {:<15} {:<12} {:<12}".format(
            "Emp ID", "Name", "Designation", "Department", "Basic", "Net"))
        print("-" * 90)
        for r in rows:
            print("{:<8} {:<20} {:<18} {:<15} {:<12} {:<12}".format(*r))
    except Error as e:
        print("Error:", e)
    finally:
        cur.close()
        con.close()

# ------------------- 3. SEARCH EMPLOYEE -------------------
def search_employee():
    con = get_connection()
    if not con:
        return
    cur = con.cursor()
    try:
        emp_id = int(input("Enter Employee ID to search: "))
        cur.execute("SELECT * FROM employee WHERE emp_id = %s", (emp_id,))
        row = cur.fetchone()
        if row:
            print("\n----- Employee Details -----")
            print(f"Emp ID       : {row[0]}")
            print(f"Name         : {row[1]}")
            print(f"Designation  : {row[2]}")
            print(f"Department   : {row[3]}")
            print(f"Phone        : {row[4]}")
            print(f"Joining Date : {row[5]}")
            print(f"Basic Salary : {row[6]}")
            print(f"DA           : {row[7]}")
            print(f"HRA          : {row[8]}")
            print(f"PF           : {row[9]}")
            print(f"Tax          : {row[10]}")
            print(f"Gross        : {row[11]}")
            print(f"Net Salary   : {row[12]}")
        else:
            print("Employee not found.")
    except Error as e:
        print("Error:", e)
    finally:
        cur.close()
        con.close()

# ------------------- 4. UPDATE EMPLOYEE -------------------
def update_employee():
    con = get_connection()
```

```
    if not con:
        return
    cur = con.cursor()
    try:
        emp_id = int(input("Enter Employee ID to update: "))
        cur.execute("SELECT * FROM employee WHERE emp_id = %s", (emp_id,))
        if not cur.fetchone():
            print("Employee not found.")
            return

        print("\nWhat do you want to update?")
        print("1. Phone")
        print("2. Basic Salary (will recalculate DA, HRA, PF, Tax, Gross, Net)")
        print("3. Department")
        choice = input("Enter choice (1/2/3): ")

        if choice == "1":
            phone = input("Enter new Phone: ")
            cur.execute("UPDATE employee SET phone = %s WHERE emp_id = %s", (phone, emp_id))
        elif choice == "2":
            basic = float(input("Enter new Basic Salary: "))
            # designation ████████ ██ ███
            cur.execute("SELECT designation FROM employee WHERE emp_id = %s", (emp_id,))
            des = cur.fetchone()[0]
            da, hra, pf, tax, gross, net = calculate_salary(basic, des)
            cur.execute("""
                UPDATE employee
                SET basic_salary=%s, da=%s, hra=%s, pf=%s, tax=%s, gross=%s, net=%s
                WHERE emp_id=%s
            """, (basic, da, hra, pf, tax, gross, net, emp_id))
        elif choice == "3":
            dept = input("Enter new Department: ")
            cur.execute("UPDATE employee SET department = %s WHERE emp_id = %s", (dept, emp_id))
        else:
            print("Invalid choice.")
            return

        con.commit()
        print("Record updated successfully!")
    except Error as e:
        print("Error:", e)
    finally:
        cur.close()
        con.close()

# ------------------- 5. DELETE EMPLOYEE -------------------
def delete_employee():
    con = get_connection()
    if not con:
        return
    cur = con.cursor()
    try:
        emp_id = int(input("Enter Employee ID to delete: "))
        cur.execute("SELECT name FROM employee WHERE emp_id = %s", (emp_id,))
        row = cur.fetchone()
        if not row:
            print("Employee not found.")
            return
        confirm = input(f"Are you sure you want to delete {row[0]}? (y/n): ")
        if confirm.lower() == "y":
            cur.execute("DELETE FROM employee WHERE emp_id = %s", (emp_id,))
            con.commit()
            print("Employee deleted successfully.")
        else:
            print("Deletion cancelled.")
    except Error as e:
        print("Error:", e)
    finally:
        cur.close()
        con.close()

# ------------------- 6. SALARY SLIP -------------------
def salary_slip():
    con = get_connection()
    if not con:
        return
    cur = con.cursor()
    try:
        emp_id = int(input("Enter Employee ID for Salary Slip: "))
        cur.execute("SELECT * FROM employee WHERE emp_id = %s", (emp_id,))
        row = cur.fetchone()
        if row:
            print("\n" + "="*50)
            print("           SALARY SLIP")
            print("="*50)
            print(f"Employee ID   : {row[0]}")
            print(f"Name          : {row[1]}")
            print(f"Designation   : {row[2]}")
            print(f"Department    : {row[3]}")
            print(f"Phone         : {row[4]}")
            print(f"Joining Date  : {row[5]}")
            print("-"*50)
            print(f"Basic Salary  : {row[6]:>12.2f}")
            print(f"DA            : {row[7]:>12.2f}")
            print(f"HRA           : {row[8]:>12.2f}")
```

```
            print(f"Gross Salary  : {row[11]:>12.2f}")
            print("-"*50)
            print(f"PF            : {row[9]:>12.2f}")
            print(f"Tax           : {row[10]:>12.2f}")
            print("-"*50)
            print(f"NET SALARY    : {row[12]:>12.2f}")
            print("="*50)
        else:
            print("Employee not found.")
    except Error as e:
        print("Error:", e)
    finally:
        cur.close()
        con.close()

# ------------------- 7. PAYROLL SUMMARY (GROUP BY) -------------------
def payroll_summary():
    con = get_connection()
    if not con:
        return
    cur = con.cursor()
    try:
        cur.execute("""
            SELECT department,
                   COUNT(*) AS emp_count,
                   MIN(net) AS min_net,
                   MAX(net) AS max_net,
                   SUM(net) AS total_net,
                   AVG(net) AS avg_net
            FROM employee
            GROUP BY department
        """)
        rows = cur.fetchall()
        if not rows:
            print("No data available.")
            return
        print("\n{:<15} {:<8} {:<12} {:<12} {:<14} {:<12}".format(
            "Department", "Count", "Min Net", "Max Net", "Total Net", "Avg Net"))
        print("-" * 80)
        for r in rows:
            print("{:<15} {:<8} {:<12.2f} {:<12.2f} {:<14.2f} {:<12.2f}".format(*r))
    except Error as e:
        print("Error:", e)
    finally:
        cur.close()
        con.close()

# ------------------- MAIN MENU -------------------
def main():
    setup_database()   # Database ■■ table ■■■ ■■■■ ■■■ ■■■■ ■■■

    while True:
        print("\n" + "="*45)
        print("     PAYROLL MANAGEMENT SYSTEM")
        print("="*45)
        print("1. Add Employee")
        print("2. View All Employees")
        print("3. Search Employee")
        print("4. Update Employee")
        print("5. Delete Employee")
        print("6. Generate Salary Slip")
        print("7. Payroll Summary (Department-wise)")
        print("8. Exit")
        print("="*45)

        choice = input("Enter your choice (1-8): ")

        if choice == "1":
            add_employee()
        elif choice == "2":
            view_all()
        elif choice == "3":
            search_employee()
        elif choice == "4":
            update_employee()
        elif choice == "5":
            delete_employee()
        elif choice == "6":
            salary_slip()
        elif choice == "7":
            payroll_summary()
        elif choice == "8":
            print("\nThank you! Exiting Payroll Management System.")
            break
        else:
            print("Invalid choice. Please try again.")

# ------------------- PROGRAM START -------------------
if __name__ == "__main__":
    main()
```